Lesson Plan: Module 3

System Integration, End-to-End Verification, and Structured Data Collection

---

An Educational Framework for AI-Driven Signal Processing

Student:	Jude Eschete
Project Advisor:	Dr. Bernard Yett
Program:	M.S. Computer Engineering
Institution:	Stevens Institute of Technology
Date Range:	March 29 – April 11, 2026

This document presents the lesson plan for the third module of the capstone project. Students connect the STM32 beacon–receiver subsystem (Module 1) with the FPGA signal processing pipeline (Module 2), diagnose and resolve integration failures that surface when individually tested subsystems operate together on real hardware, characterize end-to-end latency and throughput, and execute structured data collection campaigns that produce the labeled RF dataset for the machine learning pipeline in Module 4.

JEschete@stevens.edu

Contents

1 Module Overview **1**
1.1 Context Within the Project . . . 1
1.2 Prerequisites . . . 2

2 Weekly Schedule **3**

3 Week 1: Hardware Deployment and Integration Debugging **4**
3.1 Objectives . . . 4
3.2 Deployment Checklist . . . 4
3.3 Integration Debugging Methodology . . . 4
3.3.1 Layer 1: Physical Interface . . . 5
3.3.2 Layer 2: Baud Rate and Framing . . . 5
3.3.3 Layer 3: Protocol Synchronization . . . 5
3.3.4 Layer 4: Data Integrity . . . 5
3.4 Vivado Integrated Logic Analyzer (ILA) . . . 6
3.4.1 Recommended ILA Probe Points . . . 6
3.5 First Live Frame Verification . . . 7

4 Week 2: Performance Characterization **8**
4.1 Objectives . . . 8
4.2 Performance Metrics . . . 8
4.2.1 Packet Error Rate (PER) . . . 8
4.2.2 End-to-End Latency . . . 9
4.2.3 Sustained Throughput . . . 9
4.2.4 FIFO Peak Utilization . . . 10
4.3 Scaling Stress Test . . . 10

5 Week 3: Data Collection Campaign Design and Execution **12**
5.1 Objectives . . . 12
5.2 Experimental Design . . . 12
5.2.1 Independent Variables . . . 12
5.2.2 Experimental Matrix . . . 14
5.3 Host-Side Data Collection Script . . . 14
5.3.1 Script Architecture . . . 14
5.3.2 Feature Vector Reception Protocol . . . 14
5.3.3 CSV Output Format . . . 14
5.4 Single-Variable Campaign Execution . . . 16

6 Week 4: Multi-Variable Collection and Dataset Finalization **17**
6.1 Objectives . . . 17
6.2 Remaining Campaigns . . . 17
6.2.1 Campaign C5: Multi-Beacon Discrimination . . . 17
6.3 Dataset Merging and Cleaning . . . 18
6.4 Exploratory Data Analysis . . . 18
6.4.1 Required Visualizations . . . 19

6.5 Distribution Shift: Simulation vs. Hardware 19
6.6 Dataset Summary Document 20

7 System Architecture: Complete Integrated Pipeline 21

8 Hardware and Software Reference 22

9 Assessment Rubric 22

10 Recommended Resources 23
10.1 Datasheets and Technical References 23
10.2 Integration and Testing References 23
10.3 Data Science and Machine Learning Preparation 23
10.4 Application Notes 24

11 Transition to Module 4 25

1 Module Overview

Module 3 Learning Objectives

Upon completion of this module, the student (or future learner following this curriculum) will be able to:

LO-1 Deploy the Module 2 FPGA bitstream to the Nexys A7-100T and verify correct operation against live UART data from the STM32 receiver, identifying and resolving discrepancies between simulation behavior and hardware behavior.

LO-2 Diagnose integration failures that span disciplinary boundaries—firmware timing errors that corrupt the FPGA data stream, baud rate drift, impedance mismatches, and protocol synchronization faults—using systematic debugging techniques (ILA probes, logic analyzer captures, UART logging).

LO-3 Measure and characterize end-to-end system performance metrics: packet error rate, end-to-end latency (beacon TX to feature vector availability), sustained throughput, and FIFO utilization under load.

LO-4 Design and execute a structured data collection campaign that systematically varies beacon parameters (TX power, spreading factor, modulation type, number of concurrent beacons, physical distance) to produce a labeled RF dataset with controlled experimental conditions.

LO-5 Implement a host-side Python data collection script that receives feature vectors from the FPGA, attaches ground-truth labels, and stores the dataset in a format ready for machine learning ingestion.

LO-6 Identify and articulate the distribution shift between simulation data and hardware-collected data, understanding how real-world channel impairments, timing offsets, and hardware non-idealities produce feature distributions that differ from idealized models.

1.1 Context Within the Project

Module 3 represents the critical integration point of the curriculum. Modules 1 and 2 each developed and validated subsystems in isolation: Module 1 produced a working beacon–receiver RF subsystem with known packet formats and timing; Module 2 produced a simulated-and-verified FPGA ingestion pipeline that parses those packets and extracts feature vectors. Neither module tested whether these subsystems work correctly when connected to each other through a physical interface carrying real signals.

This gap is intentional. The framework's central pedagogical thesis is that cross-domain integration failures are the primary subject of instruction, not incidental obstacles. A firmware DMA timing error that has no observable effect during Module 1 standalone testing may cause byte-level misalignment in the FPGA parser. A CRC engine that passes every simulation testbench in Module 2 may reject valid frames on hardware due to a byte-ordering mismatch in the STM32 firmware. These failures require students to reason simultaneously across firmware, hardware interface, and digital logic domains—the exact multidisciplinary skill the framework is designed to develop.

The second half of this module shifts from integration debugging to structured data collection.

The validated end-to-end pipeline becomes a scientific instrument: students use it to execute controlled experiments that vary beacon parameters and collect the labeled RF feature dataset that drives all machine learning work in Module 4. The quality, diversity, and labeling integrity of this dataset directly determine what classifiers can be trained and how well they generalize. This dependency makes the data collection campaign a non-trivial engineering exercise, not merely a "run the system and save files" activity.

1.2 Prerequisites

Students beginning this module should have:

- Completed Module 1 with a working multi-beacon RF subsystem (at least 2 beacons + 1 receiver, all validated for packet transmission and reception).
- Completed Module 2 with a fully simulated FPGA pipeline (UART RX → parser → CRC → feature extraction → BRAM), a generated bitstream, and a verified constraint file (`.xdc`).
- All hardware assembled and wired: STM32 receiver UART TX connected to Nexys A7 PMOD JA, FPGA USB-UART connected to host PC, beacons powered and antennas attached.
- Python 3.10+ with `pyserial` installed on the host machine.
- Familiarity with Vivado ILA (Integrated Logic Analyzer) for on-chip signal probing.
- Access to a logic analyzer (optional but recommended for Week 1 debugging).

2 Weekly Schedule

The module spans two weeks (four working sessions). The first week focuses on hardware deployment, integration debugging, and performance characterization. The second week focuses on structured data collection and dataset preparation.

Table 1: Module 3 weekly breakdown.

Week	Focus Area	Key Activities
1	Hardware Deployment & Integration Debugging	Program FPGA with Module 2 bitstream; connect STM32 receiver; diagnose and resolve integration failures (baud mismatch, byte alignment, CRC rejects); verify first live frame through the complete pipeline.
2	Performance Characterization	Measure end-to-end latency, throughput, and packet error rate; stress-test with multiple beacons; identify bottlenecks; tune FIFO depths and export timing; document system performance envelope.
3	Data Collection Campaign Design & Execution	Design experimental matrix (TX power, SF, modulation, distance, beacon count); implement host-side Python collection script; execute single-variable sweep campaigns; validate dataset integrity and label correctness.
4	Multi-Variable Collection & Dataset Finalization	Execute multi-variable and edge-case collection runs; merge and clean dataset; perform exploratory data analysis; document dataset structure, provenance, and known limitations for Module 4 handoff.

3 Week 1: Hardware Deployment and Integration Debugging

3.1 Objectives

- Program the Nexys A7-100T with the Module 2 bitstream and verify basic I/O (LEDs, push-buttons, seven-segment display).
- Connect the STM32 receiver's UART TX to the FPGA's PMOD input and observe the first live data on the FPGA.
- Diagnose and resolve integration failures using a systematic debugging methodology.
- Achieve the first successfully parsed and CRC-verified frame on live hardware.

3.2 Deployment Checklist

Before connecting subsystems, each component must be verified independently on the bench:

Table 2: Pre-integration hardware checklist.

#	Check	Pass?
1	Nexys A7 powers on via USB; Vivado Hardware Manager detects the FPGA.	☐
2	Module 2 bitstream programs successfully; LED test pattern confirms I/O.	☐
3	FPGA USB-UART loopback: characters typed in terminal echo back correctly at 115200 baud.	☐
4	STM32 receiver firmware loaded; beacon packets print to STM32 debug UART.	☐
5	Beacon A transmitting (LED blink or debug UART confirms TX cycle).	☐
6	Beacon B transmitting (same verification).	☐
7	Receiver reports both beacon IDs in debug log with valid RSSI/SNR.	☐
8	Physical wiring: receiver UART TX → PMOD JA pin 1; GND connected between boards.	☐

Critical: Common Ground

The STM32 Nucleo and Nexys A7 are powered from separate USB ports and therefore have separate ground references. A dedicated GND jumper wire between the Nucleo GND pin and the PMOD JA GND pin is **mandatory**. Without a common ground, the UART signal levels are referenced to different potentials and the FPGA will read garbage. This is the single most common integration failure in multi-board UART connections and is trivially fixed but surprisingly easy to overlook.

3.3 Integration Debugging Methodology

Integration failures fall into predictable categories. The following systematic approach isolates the root cause efficiently:

3.3.1 Layer 1: Physical Interface

- **Symptom:** FPGA sees no transitions on `uart_rxd`; LED frame counter stays at zero.
- **Diagnosis:** Probe the PMOD pin with an oscilloscope or logic analyzer. If no activity is visible, the wiring or pin assignment is wrong. Verify the STM32 TX pin mapping, the PMOD pin number, and the `.xdc` constraint file entry.
- **Common causes:** TX/RX swap (receiver TX connected to wrong PMOD pin); missing GND jumper; PMOD pin constrained to wrong FPGA ball in `.xdc`; STM32 UART TX not configured on the expected GPIO.

3.3.2 Layer 2: Baud Rate and Framing

- **Symptom:** FPGA sees transitions but `rx_error` (framing error) asserts frequently; parsed bytes are incorrect.
- **Diagnosis:** Use a logic analyzer to capture the UART waveform and manually verify the bit timing. Measure the actual bit period and compare to the expected 8.68 μs. Calculate the baud rate deviation.

$$\text{Deviation (\%)} = \frac{|T_{\text{measured}} - T_{\text{expected}}|}{T_{\text{expected}}} \times 100 \tag{1}$$

  The Module 2 UART receiver tolerates $\pm 3\%$ with $16\times$ oversampling. If the deviation exceeds this, the STM32's baud rate prescaler must be adjusted.
- **Common causes:** STM32 system clock misconfigured (e.g., running at 72 MHz instead of 80 MHz, producing a different UART divisor); FPGA baud rate generic set incorrectly; stop bit configuration mismatch (STM32 sending 2 stop bits, FPGA expecting 1).

3.3.3 Layer 3: Protocol Synchronization

- **Symptom:** Bytes arrive correctly (no framing errors) but the parser never asserts `frame_valid`; it remains stuck in HUNT state.
- **Diagnosis:** Use the Vivado ILA to capture the byte stream entering the parser. Look for the `0x7E` start delimiter. If it never appears, the receiver firmware may not be emitting the framing format expected by the parser (e.g., sending raw beacon packets without the receiver forwarding frame wrapper defined in Module 1 Table 6).
- **Common causes:** Receiver firmware updated since Module 1 with a different frame format; start delimiter value changed; STM32 byte ordering (endianness) differs from FPGA expectation.

3.3.4 Layer 4: Data Integrity

- **Symptom:** Parser finds `0x7E` and progresses through states, but CRC-8 rejects every frame.
- **Diagnosis:** Capture the complete 43-byte frame in the ILA. Manually compute the CRC-8 over bytes 1–41 and compare to byte 42. If the manual CRC matches byte 42 but the hardware CRC does not, the VHDL CRC implementation has a bug (likely initialization or

byte-ordering). If the manual CRC does *not* match byte 42, the STM32 firmware is computing the CRC differently (different polynomial, different initialization, or different byte range).

- **Common causes:** CRC polynomial mismatch between firmware and RTL; CRC initialization value mismatch; firmware includes the start delimiter in the CRC computation but the RTL does not (or vice versa); endianness disagreement on multi-byte fields.

Design Principle: Integration Failures Are Learning Objectives

The debugging categories above are not a list of mistakes to avoid—they are the *subject matter* of this module. Each failure mode requires the student to reason across the firmware/hardware boundary and understand how a decision in one domain (e.g., the STM32's CRC initialization value) propagates requirements into another domain (the VHDL CRC module). The framework deliberately stages development so that these mismatches are likely to occur: Module 1 and Module 2 are developed in partial isolation precisely so that integration surfaces real cross-domain issues. A curriculum that pre-solves these mismatches would eliminate the most valuable learning experiences.

3.4 Vivado Integrated Logic Analyzer (ILA)

The ILA is the primary on-chip debugging tool for FPGA integration. It captures internal signal transitions into block RAM and streams the captured data to Vivado's waveform viewer over JTAG.

3.4.1 Recommended ILA Probe Points

Table 3: ILA probe signals for integration debugging.

Signal	Width	Purpose
`uart_rxd` (synchronized)	1	Verify serial transitions are reaching FPGA
`rx_data[7:0]`	8	Decoded bytes from UART receiver
`rx_valid`	1	Byte-level timing; verify one pulse per byte
`rx_error`	1	Framing error detection
`parser_state[2:0]`	3	Parser FSM state (HUNT, LENGTH, META, etc.)
`byte_counter[5:0]`	6	Position within current frame
`frame_valid`	1	Trigger: successful frame reception
`frame_reject`	1	Trigger: CRC failure
`beacon_id[7:0]`	8	Parsed beacon ID (verify correct extraction)
`rssi[7:0]`	8	Parsed RSSI (compare to STM32 debug log)
`fifo_count[5:0]`	6	FIFO occupancy (detect overflow risk)

ILA Resource Cost

Each ILA probe consumes block RAM for the capture buffer. A typical configuration—12 probes, 1024-sample depth—uses approximately 4 BRAM tiles. This is acceptable during integration debugging (the Module 2 pipeline uses only 5 of 270 available BRAM tiles) but should be removed from the final production bitstream to free resources. Mark ILA instantiations with a generate conditional controlled by a top-level generic (`g_DEBUG : boolean := true`) so they can be excluded without editing RTL.

3.5 First Live Frame Verification

Exercise: First Frame on Hardware

With one beacon transmitting at 2-second intervals and the receiver connected to the FPGA:

1. Program the FPGA and open the ILA dashboard in Vivado.
2. Set the ILA trigger to `frame_valid` rising edge.
3. Arm the trigger and wait for a capture.
4. When a frame is captured, verify the following against the STM32 receiver debug log:
   - The parsed Beacon ID matches the transmitting beacon's configured ID.
   - The parsed RSSI is within ±2 dB of the value reported by the STM32.
   - The sequence number matches the beacon's current count.
   - Both CRC checks pass (`frame_valid` = '1', `beacon_crc_ok` = '1').
5. If the trigger never fires (no `frame_valid`), set the trigger to `rx_valid` rising edge instead and verify that bytes are arriving. Work through the debugging layers above.

Milestone: The first successfully parsed live frame represents a validated end-to-end data path spanning all four hardware components (beacon MCU → RF link → receiver MCU → FPGA fabric).

Week 1 Deliverable

Integration Debug Report — A document containing:

1. Pre-integration checklist results (Table 2), with photos of the physical setup.
2. Log of all integration issues encountered, categorized by debugging layer (physical, baud rate, protocol, data integrity), with root cause and resolution for each.
3. ILA waveform captures showing: first successful frame reception, parser state transitions during a live frame, and at least one diagnosed failure (before the fix was applied).
4. Side-by-side comparison of FPGA-parsed field values vs. STM32 debug log values for 10 consecutive frames.

4 Week 2: Performance Characterization

4.1 Objectives

- Define and measure the four primary performance metrics: packet error rate (PER), end-to-end latency, sustained throughput, and FIFO peak utilization.
- Stress-test the pipeline with increasing beacon counts to identify the scaling limit.
- Identify performance bottlenecks and, where possible, mitigate them.
- Establish the system's performance envelope for the data collection campaigns in Weeks 3–4.

4.2 Performance Metrics

4.2.1 Packet Error Rate (PER)

The packet error rate captures the fraction of transmitted beacon packets that fail to produce a valid feature vector at the FPGA output. It incorporates losses at every stage: RF link (missed receptions), UART transport (framing errors), and CRC verification (corrupted frames).

$$\text{PER} = 1 - \frac{N_{\text{valid features}}}{N_{\text{transmitted packets}}} \tag{2}$$

PER is measured by comparing the FPGA's observation count for each beacon (stored in the per-beacon state RAM) against the beacon's own sequence number. If Beacon A has transmitted 500 packets (sequence number = 500) and the FPGA reports an observation count of 487, then $\text{PER}_A = 1 - 487/500 = 2.6\%$.

Decomposing PER

A single PER number is useful for system-level characterization but insufficient for debugging. Students should decompose PER into its contributing factors:

- **RF loss:** Packets the receiver never detected (compare receiver log count to beacon sequence number).
- **UART loss:** Packets the receiver detected but the FPGA UART receiver missed (framing errors counted on the FPGA).
- **CRC loss:** Packets the FPGA received but rejected due to CRC failure (`frame_reject` counter).
- **FIFO overflow loss:** Packets lost because the receive FIFO was full when a new byte arrived (overflow counter).

This decomposition reveals *where* in the pipeline packets are lost, which directly informs optimization decisions. An RF-dominated PER suggests environmental or link budget issues (Module 1 domain); a CRC-dominated PER suggests a firmware/RTL mismatch (cross-domain); a FIFO-dominated PER suggests the FPGA pipeline cannot keep up with the data rate (Module 2 domain).

4.2.2 End-to-End Latency

End-to-end latency is the time from beacon packet transmission to feature vector availability in the FPGA BRAM. It comprises:

$$L_{\text{e2e}} = L_{\text{RF}} + L_{\text{RX_FW}} + L_{\text{UART}} + L_{\text{FPGA}} \tag{3}$$

Table 4: Latency budget breakdown.

Component	Estimated	Description
L_{RF}	50–500 ms	LoRa on-air time (depends on SF: SF7 ≈ 50 ms, SF12 ≈ 1.5 s for 32-byte payload)
$L_{\text{RX_FW}}$	<1 ms	Receiver ISR + FIFO read + metadata extraction + frame construction
L_{UART}	3.73 ms	43 bytes × 10 bits/byte ÷ 115200 baud
L_{FPGA}	<0.01 ms	Pipeline processing at 100 MHz (∼6 clock cycles after last byte)
Total (SF7)	**∼55 ms**	Dominated by RF on-air time
Total (SF12)	**∼1505 ms**	Dominated by RF on-air time

Measuring Latency on Hardware

Direct measurement of L_{e2e} requires correlating timestamps across two asynchronous clock domains (STM32 system tick and FPGA clock). Two practical approaches:

1. **GPIO toggle method:** The beacon asserts a GPIO pin when it begins a TX. The FPGA asserts an output pin when `frame_valid` fires. Both signals are captured on an oscilloscope or logic analyzer, and the time difference is L_{e2e}.

2. **Timestamp difference method:** Compare the beacon-side timestamp (embedded in the packet, reflecting the STM32 tick at TX time) against the receiver-side RX timestamp (also an STM32 tick). This measures $L_{\text{RF}} + L_{\text{RX_FW}}$ but not the UART and FPGA components. Adding the fixed UART transfer time gives a good approximation.

For the purposes of this module, the GPIO toggle method is preferred because it provides a true end-to-end measurement without relying on timestamp synchronization between boards.

4.2.3 Sustained Throughput

Throughput is the maximum rate at which the pipeline can produce valid feature vectors, expressed in frames per second (fps). The theoretical maximum is constrained by the UART link:

$$\text{Throughput}_{\text{max}} = \frac{\text{Baud Rate}}{\text{Bits per Frame}} = \frac{115200}{43 \times 10} = 268 \text{ fps} \tag{4}$$

In practice, the achievable throughput will be lower due to inter-frame gaps from the STM32 firmware and RF on-air time between transmissions. The important measurement is whether the

FPGA pipeline can sustain the frame rate imposed by the beacon configuration without FIFO overflow.

4.2.4 FIFO Peak Utilization

The receive FIFO (64 entries, Module 2 Week 1) provides back-pressure buffering. Under normal two-beacon operation the FIFO should rarely exceed 50% occupancy. However, as beacon count increases or transmissions cluster (ALOHA mode collisions followed by burst receptions), the FIFO may approach capacity. Students should monitor `fifo_count` via the ILA or map it to the LEDs to observe real-time utilization.

4.3 Scaling Stress Test

Exercise: Beacon Count Scaling

Progressively increase the number of active beacons and record performance at each step:

Beacons	Mode	Interval	Expected fps	Measured PER
1	Fixed, 2 s	2 s	0.5	
2	Fixed, 2 s / 3 s	staggered	∼0.83	
2	ALOHA, [1,3] s	random	∼1.0	
3	Fixed, 1 s each	staggered	3.0	
3	ALOHA, [0.5,2] s	random	∼3.0–4.0	

For each configuration:

1. Run for 5 minutes.
2. Record: total packets transmitted (from beacon sequence numbers), total valid features (from FPGA observation counts), CRC reject count, FIFO peak utilization, framing error count.
3. Compute PER and decompose it.
4. If FIFO peak exceeds 75%, flag this as a throughput risk and propose a mitigation (increase FIFO depth, upgrade to SPI, reduce inter-frame overhead in receiver firmware).

Week 2 Deliverable

System Performance Characterization Report — A document containing:

1. PER measurements for each beacon configuration tested, with per-stage loss decomposition.
2. End-to-end latency measurement results (method, raw measurements, computed averages and standard deviation) for at least two spreading factor settings (SF7 and SF10).
3. Throughput analysis: theoretical maximum vs. measured sustained rate.
4. FIFO utilization traces for the stress-test configurations.
5. Identification of the system's throughput bottleneck and a proposed mitigation plan (even if not implemented).
6. Summary table of the system's performance envelope, establishing the operating parameters for the data collection campaign.

5 Week 3: Data Collection Campaign Design and Execution

5.1 Objectives

- Design a structured experimental matrix that systematically varies beacon parameters to produce a diverse, labeled dataset.
- Implement a host-side Python script that receives feature vectors from the FPGA via UART and stores them with ground-truth labels.
- Execute single-variable sweep campaigns and validate dataset integrity.
- Understand the concept of distribution shift and how hardware-collected data differs from simulation.

5.2 Experimental Design

The data collection campaign is designed as a controlled experiment: one independent variable is swept at a time while all other parameters are held constant. This single-variable approach produces a dataset where the effect of each parameter on the feature vector is isolable—a critical property for understanding ML model behavior in Module 4.

5.2.1 Independent Variables

Table 5: Experimental independent variables and sweep ranges.

Variable	Range	Steps	ML Relevance
TX Power	+2 to +20 dBm	4 (2, 8, 14, 20)	Affects RSSI distribution; tests classifier robustness to signal strength variation
Spreading Factor	SF7 – SF12	6 (each SF)	Primary classification target; each SF produces different on-air waveform and timing
Modulation Type	LoRa, FSK	2	Second classification target; fundamentally different signal structure
Beacon Count	1, 2, 3	3	Tests multi-emitter discrimination; increases collision probability
Physical Distance	1 m, 3 m, 5 m, 8 m	4	Affects RSSI, SNR, and frequency error; introduces path loss variation

Table 6: Single-variable sweep campaign plan.

ID	Sweep Variable	Fixed Parameters	Packets	Duration	Notes
C1	TX Power (4 levels)	SF7, LoRa, 1 beacon, 3 m	200/level	∼27 min	Baseline power sweep
C2	Spreading Factor (6)	+14 dBm, LoRa, 1 beacon, 3 m	200/SF	∼80 min	SF classification dataset
C3	Modulation (LoRa/FSK)	+14 dBm, 1 beacon, 3 m	300/type	∼20 min	Binary modulation classification
C4	Distance (4 levels)	+14 dBm, SF7, LoRa, 1 beacon	200/dist	∼27 min	Path loss characterization
C5	Beacon Count (1/2/3)	+14 dBm, SF7, LoRa, 3 m	300/config	∼50 min	Multi-emitter discrimination

5.2.2 Experimental Matrix

Design Principle: Packets Per Configuration

The 200–300 packets per configuration is a deliberate trade-off between dataset size and collection time. For simple classifiers (random forest, SVM), 200 samples per class is typically sufficient for a benchtop dataset with low intra-class variance. For deep learning models (CNN, MLP), more data is always beneficial, but the primary goal of Module 4 is to demonstrate the ML pipeline, not to achieve state-of-the-art accuracy. Students who find time can increase the sample count during Week 4 multi-variable runs.

Each configuration should produce enough samples to split into training (70%), validation (15%), and test (15%) subsets with at least 30 samples in the smallest test split—which 200 samples per class comfortably supports.

5.3 Host-Side Data Collection Script

The Python collection script runs on the host PC, receives feature vectors from the FPGA over the USB-UART interface, attaches ground-truth labels from the experimental configuration, and writes records to a CSV file.

5.3.1 Script Architecture

5.3.2 Feature Vector Reception Protocol

The FPGA's UART TX export module (Module 2 Week 3) transmits feature vectors using a simple framed protocol:

Table 7: FPGA-to-host feature export frame (35 bytes).

Field	Offset	Size	Description
Start Marker	0	1 B	`0xFE` (distinct from the `0x7E` receiver frame delimiter)
Feature Vector	1	32 B	Complete feature vector (Module 2 Table 5 format)
CRC-8	33	1 B	CRC-8 over bytes 1–32 (transport integrity)
End Marker	34	1 B	`0xFF` (frame terminator)

5.3.3 CSV Output Format

Each row in the output CSV represents one received feature vector with its experimental context:

Table 8: Dataset CSV column schema.

Column	Source	Description
`campaign_id`	Config JSON	Experiment identifier (C1, C2, etc.)
`timestamp_host`	Host clock	Host-side reception time (ISO 8601)
`beacon_id`	Feature vector	FPGA-parsed beacon ID
`mod_code`	Feature vector	Modulation code
`spread_factor`	Feature vector	Spreading factor
`bw_code`	Feature vector	Bandwidth code
`tx_power`	Feature vector	Transmit power (dBm)
`rssi`	Feature vector	Received signal strength (dBm)
`snr`	Feature vector	Signal-to-noise ratio (dB)
`freq_error`	Feature vector	Carrier frequency error (Hz)
`rx_timestamp`	Feature vector	Receiver-side timestamp (ms)
`beacon_timestamp`	Feature vector	Beacon-side timestamp (ms)
`seq_num`	Feature vector	Sequence number
`payload_crc_ok`	Feature vector	Beacon CRC status (1/0)
`inter_arrival_ms`	Feature vector	FPGA-computed inter-arrival time
`rssi_delta`	Feature vector	FPGA-computed RSSI change
`obs_count`	Feature vector	FPGA-computed observation count
`distance_m`	Config JSON	Physical beacon–receiver distance (m)
`label_sf`	Config JSON	Ground-truth spreading factor
`label_mod`	Config JSON	Ground-truth modulation type
`label_beacon`	Config JSON	Ground-truth beacon identity

Label Integrity

- **Self-reported vs. ground-truth labels:** The feature vector contains the beacon's *self-reported* parameters (modulation code, SF, TX power). The campaign config provides the *experimentally controlled* ground-truth labels. In a well-functioning system these match. When they do not—a beacon claiming SF7 while configured for SF10—it indicates a firmware bug or configuration error. The collection script should log mismatches as warnings without discarding the record, as these mismatches are themselves informative for Module 4 analysis.

- **Distance labels are manual:** Physical distance is measured by the student and entered in the campaign config. Measurement error is unavoidable; record distance to the nearest 0.5 m and note the measurement method.

- **No label leakage:** The `label_*` columns are derived from the experimental setup, not from the feature vector fields. In Module 4, students must be careful not to use the self-reported fields as prediction targets when the same information is also a model input—this would create trivial classification with no generalization value.

5.4 Single-Variable Campaign Execution

Exercise: TX Power Sweep (Campaign C1)

Execute the first data collection campaign:

1. Configure one beacon at 3 m distance, SF7, LoRa modulation, 2-second fixed interval.
2. Set TX power to +2 dBm. Start the collection script with `campaign_id = "C1"` and `tx_power_label = 2`.
3. Collect 200 packets (approximately 7 minutes). Stop the script.
4. Repeat at +8 dBm, +14 dBm, and +20 dBm.
5. After all four runs, open the merged CSV and verify:
   - 800 total rows (4×200).
   - RSSI values increase monotonically with TX power (on average).
   - SNR values are positive at all power levels (confirming adequate link margin at 3 m).
   - Sequence numbers are monotonically increasing within each run.
   - No label mismatches between self-reported TX power and `tx_power_label`.

Week 3 Deliverable

Data Collection Campaign Report (Part 1) — A document containing:

1. Experimental matrix design with justification for variable ranges and sample counts.
2. Python collection script source code with inline documentation.
3. Campaign C1 (TX Power sweep) results: summary statistics (mean, std) for RSSI and SNR at each power level, scatter plot of RSSI vs. TX Power.
4. Campaign C2 (Spreading Factor sweep) results: summary statistics, box plots of inter-arrival time by SF (should increase with SF due to longer on-air time).
5. Preliminary assessment of data quality: label mismatches, CRC failures, missing sequence numbers.

6 Week 4: Multi-Variable Collection and Dataset Finalization

6.1 Objectives

- Execute the remaining single-variable campaigns (C3 – C5) and any additional multi-variable runs.
- Merge all campaign CSV files into a unified dataset with consistent schema and verified label integrity.
- Perform exploratory data analysis (EDA) to understand feature distributions and correlations.
- Document the dataset structure, provenance, and known limitations.
- Identify and articulate the distribution shift between simulation-derived data and hardware-collected data.

6.2 Remaining Campaigns

Complete campaigns C3 (modulation sweep), C4 (distance sweep), and C5 (beacon count scaling). For each campaign, follow the same collection protocol as C1/C2: configure parameters, start the script, collect the target packet count, stop, and validate.

6.2.1 Campaign C5: Multi-Beacon Discrimination

Campaign C5 is the most complex, as it requires coordinating multiple beacons and interpreting interleaved data:

Table 9: Campaign C5 configurations.

Config	Beacons	Mode	Interval	Purpose
C5a	1	Fixed	2 s	Baseline (no interference)
C5b	2	Fixed	2 s / 3 s	Staggered (minimal collision)
C5c	2	ALOHA	[1,3] s	Random access (some collisions)
C5d	3	Fixed	1 s each	High-rate staggered
C5e	3	ALOHA	[0.5,2] s	Maximum contention

What to Watch For in C5

- **PER increase:** Under ALOHA mode, packet collisions cause RF-level losses. Compare PER between fixed-interval (near-zero collision) and ALOHA (collision-prone) configurations.
- **RSSI variation:** With multiple beacons, the receiver may capture packets from different beacons in rapid succession, producing interleaved RSSI values. The RSSI Delta feature becomes more volatile.
- **Inter-arrival time distribution:** Under fixed-interval scheduling, inter-arrival time for each beacon should be tightly clustered around the configured interval. Under ALOHA, it should follow the configured uniform distribution. These distributional differences are features that an ML classifier could exploit for scheduling-mode classification.

6.3 Dataset Merging and Cleaning

After all campaigns are complete, merge the individual CSV files into a single dataset. The merging script should:

1. Concatenate all campaign CSVs, preserving the `campaign_id` column for provenance tracking.
2. Verify schema consistency (all files have the same columns in the same order).
3. Remove duplicate rows (same beacon ID, same sequence number, same campaign—indicates a script restart that re-collected data).
4. Flag rows where self-reported parameters disagree with ground-truth labels (do not remove—flag and count).
5. Add a `split` column assigning each row to `train` (70%), `val` (15%), or `test` (15%) using stratified random sampling by `campaign_id` and `label_sf`.

Stratified Splitting

The train/val/test split must be stratified by the classification target to ensure each split contains representative samples from every class. For the SF classification task, stratify by `label_sf`. For the modulation classification task, stratify by `label_mod`. If the dataset will be used for both tasks, stratify by the conjunction (e.g., group by (SF, mod) pairs). Failure to stratify can produce a test set with zero samples for a minority class, making evaluation meaningless.
Additionally, the split must respect **temporal ordering within campaigns**: samples from the same campaign run should not be split across train and test sets, as temporally adjacent samples are correlated (similar RSSI, similar channel conditions). Split at the campaign-run level, not the individual-sample level.

6.4 Exploratory Data Analysis

Before handing the dataset to Module 4, students should perform basic EDA to understand the data they collected:

6.4.1 Required Visualizations

1. **RSSI distribution by TX power:** Histogram or violin plot showing the RSSI distribution for each TX power level in Campaign C1. Verify that distributions shift right (stronger signal) as TX power increases.

2. **RSSI vs. distance:** Scatter plot from Campaign C4 showing RSSI as a function of distance. Overlay the theoretical free-space path loss (FSPL) curve from Module 1 and discuss deviations (indoor multipath, obstruction).

$$\mathrm{RSSI}_{\mathrm{theoretical}}(d) = P_{\mathrm{tx}} + G_{\mathrm{tx}} + G_{\mathrm{rx}} - 20\log_{10}(d) - 20\log_{10}(f) + 147.55 - L_{\mathrm{misc}} \tag{5}$$

3. **Feature correlation matrix:** Compute and visualize the Pearson correlation between all numeric features. Identify highly correlated feature pairs (e.g., TX power and RSSI should be strongly correlated; RSSI and SNR may also correlate).

4. **Inter-arrival time distribution by scheduling mode:** Compare the inter-arrival time histograms for Campaign C5b (fixed) vs. C5c (ALOHA). The fixed-interval distribution should be tightly peaked; the ALOHA distribution should be broader and approximately uniform.

5. **Class balance:** Bar chart showing the number of samples per class for each classification target (SF, modulation, beacon ID). Verify that no class is severely underrepresented.

6.5 Distribution Shift: Simulation vs. Hardware

Design Principle: Distribution Shift as a Teaching Moment

The engagement simulator developed early in the project generates synthetic labeled data in a controlled physics model. The hardware-collected dataset from this module captures real RF signals through physical hardware. These two data sources will not have identical feature distributions, even for equivalent parameter settings. This discrepancy—distribution shift—is one of the most important practical lessons in applied machine learning.
Students should compare:

- **Simulator RSSI** vs. **hardware RSSI** at equivalent distance and TX power. The simulator uses a clean FSPL model; hardware RSSI includes multipath fading, antenna pattern irregularity, and receiver front-end non-linearities.

- **Simulator inter-arrival time** vs. **hardware inter-arrival time**. The simulator's tick-based engine produces exact timing; hardware timing includes STM32 interrupt jitter, RF channel contention, and UART buffering delays.

- **Simulator SNR** (if modeled) vs. **hardware SNR**. The simulator's noise model is typically additive white Gaussian; real indoor environments produce colored noise, interference, and bursty error patterns.

The key takeaway for Module 4 is: a classifier trained on simulation data may not generalize to hardware data (and vice versa). Documenting this shift here gives the ML work a clear empirical foundation.

6.6 Dataset Summary Document

The dataset summary document is the data contract between Module 3 (data producers) and Module 4 (data consumers). It must contain everything an ML practitioner needs to understand, load, and correctly use the dataset without referring to Module 3 implementation details.

Week 4 Deliverable

Dataset and Data Collection Report (Part 2) — A document containing:

1. Completed campaign results for C3, C4, and C5, with summary statistics and any anomalies.
2. Merged dataset file (`.csv`) with consistent schema, provenance columns, and stratified train/val/test split.
3. Dataset summary statistics: total rows, rows per campaign, rows per class (SF, modulation, beacon ID), label mismatch count, CRC failure count.
4. EDA visualizations (all 5 required plots described above) with interpretive captions.
5. Distribution shift analysis: at least two quantitative comparisons between simulation data and hardware-collected data (e.g., RSSI mean/std, inter-arrival time histogram divergence).
6. Known limitations: environmental factors (indoor multipath, ambient interference), measurement uncertainties (distance accuracy, manual reconfiguration between campaigns), and any collection anomalies (power interruptions, firmware crashes, beacon desynchronization).
7. Data dictionary: complete column descriptions, units, data types, and valid ranges for every field in the CSV schema.

7 System Architecture: Complete Integrated Pipeline

Figure 2 shows the complete system architecture as validated by the end of Module 3. All blocks are now physical hardware connected by real interfaces, and the data flow has been characterized end-to-end.

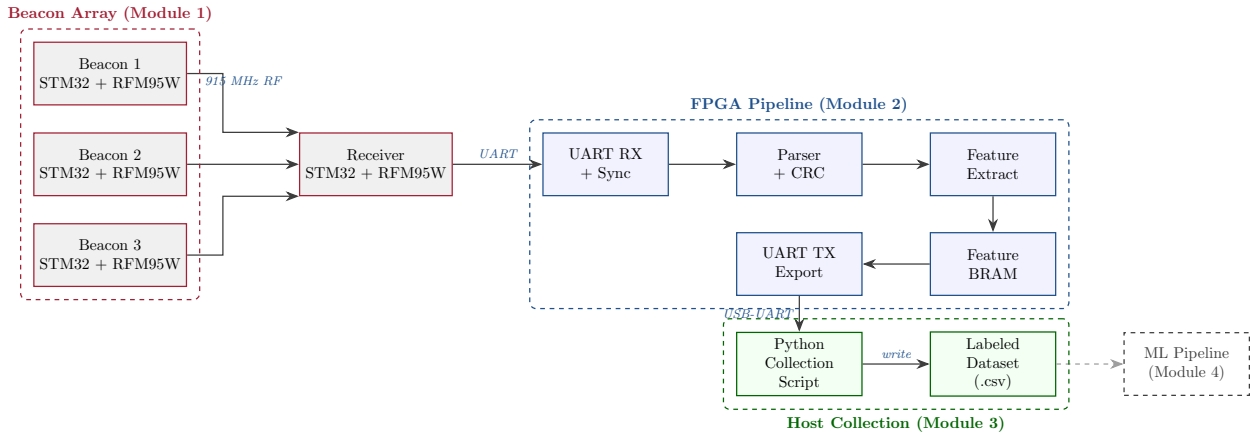

Figure 2: Complete integrated system architecture at the end of Module 3. All connections are physical hardware interfaces validated end-to-end. The labeled dataset (green) is the primary output of this module and the input to Module 4.

8 Hardware and Software Reference

Table 10: Required hardware and software for Module 3.

Item	Qty / Version	Purpose
STM32 Nucleo-L476RG	≥ 3	2 beacon MCUs + 1 receiver MCU
RF Transceiver Module (RFM95W)	≥ 3	Beacon and receiver RF communication
915 MHz Whip Antenna	≥ 3	RF transmission/reception
Digilent Nexys A7-100T	1	FPGA processing platform
USB Micro-B Cables	4–5	Power, programming, and UART communication
Breadboard + Jumper Wires	—	Inter-board connections (UART, GND)
Logic Analyzer (optional)	1	UART waveform analysis during debugging
Tape Measure	1	Physical distance measurement for Campaign C4
AMD Vivado Design Suite	2024.1+	Bitstream generation and ILA debugging
STM32CubeIDE	Latest	Beacon/receiver firmware modification
Python 3.10+	—	Host-side collection script and EDA
`pyserial`	$\geq$3.5	UART communication from Python
`pandas`	$\geq$2.0	Dataset manipulation and merging
`matplotlib` + `seaborn`	—	EDA visualizations
Terminal Emulator	—	UART debug and monitoring

9 Assessment Rubric

Table 11: Module 3 assessment rubric.

Assessment Item	Weight	Due
Integration Debug Report (checklist, issue log, ILA captures, field comparison)	20%	Week 1
System Performance Characterization Report (PER, latency, throughput, scaling)	25%	Week 2
Data Collection Campaign Report Part 1 (experimental design, script, C1/C2 results)	25%	Week 3
Dataset and Data Collection Report Part 2 (C3–C5, merged dataset, EDA, distribution shift, data dictionary)	30%	Week 4

10 Recommended Resources

10.1 Datasheets and Technical References

1. Semtech, "SX1276/77/78/79 Datasheet," Rev. 7, 2020. (Register map for RSSI, SNR, and frequency error readback.)
2. STMicroelectronics, "STM32L476xx Reference Manual," RM0351, Rev. 9, 2023. (UART peripheral timing, DMA configuration.)
3. Digilent, "Nexys A7 Reference Manual," Rev. C, 2022. (PMOD pinout, USB-UART bridge specifications.)
4. Xilinx/AMD, "Vivado Design Suite User Guide: Programming and Debugging," UG908, 2023. (ILA configuration, trigger setup, waveform capture.)

10.2 Integration and Testing References

1. P. P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx Artix-7 Edition*, 2nd ed., Wiley, 2017. (Chapter on hardware verification and ILA usage.)
2. J. W. Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, 6th ed., 2023. (UART debugging techniques, logic analyzer usage.)

10.3 Data Science and Machine Learning Preparation

1. W. McKinney, *Python for Data Analysis*, 3rd ed., O'Reilly, 2022. (pandas, data cleaning, exploratory data analysis.)
2. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O'Reilly, 2022. (Chapter 2: dataset splitting, stratified sampling, data exploration.)
3. PySerial Documentation. [Online]. Available: https://pyserial.readthedocs.io/

10.4 Application Notes

1. Semtech AN1200.22, "LoRa Modulation Basics." (On-air time calculation for different SF/BW configurations.)

2. Semtech AN1200.13, "SX1276/7/8 LoRa Modem Design Guide." (Link budget, sensitivity, and interference behavior.)

11 Transition to Module 4

Upon successful completion of Module 3, students will have:

- A fully integrated and validated end-to-end pipeline: beacon RF transmission → STM32 reception → FPGA ingestion and feature extraction → host-side data collection.
- Characterized system performance with quantified PER, latency, throughput, and scaling behavior across multiple beacon configurations.
- Diagnosed and resolved real cross-domain integration failures, gaining direct experience with the interface problems that span firmware, hardware, and digital logic boundaries.
- A labeled RF dataset of approximately 2,500–3,500 feature vectors spanning 5 experimental campaigns with controlled variation of TX power, spreading factor, modulation type, distance, and beacon count.
- A host-side Python collection script and dataset pipeline that can be re-run to collect additional data if needed.
- Exploratory data analysis documenting feature distributions, correlations, and class balance.
- A quantitative understanding of the distribution shift between simulation data and hardware-collected data.

Module 4 will consume this dataset directly. Students will preprocess the FPGA-extracted features, train and evaluate classifiers for spreading factor detection, modulation classification, and emitter identification, and analyze how classification accuracy varies as a function of SNR, beacon count, and physical distance. The dataset's controlled experimental structure (single-variable sweeps with known ground truth) enables students to isolate which features drive model performance and which real-world factors degrade accuracy compared to simulation—questions that can only be answered with the kind of carefully collected, labeled hardware data this module produces.

The distribution shift analysis from this module's EDA provides the empirical foundation for a key Module 4 experiment: training a classifier on simulation data (from the engagement simulator) and evaluating it on hardware data (from this module), then observing the accuracy drop as a direct measurement of the sim-to-real gap. This experiment is only possible because the framework stages data collection *before* ML development, giving students both data sources to compare.

End of Module 3 Lesson Plan